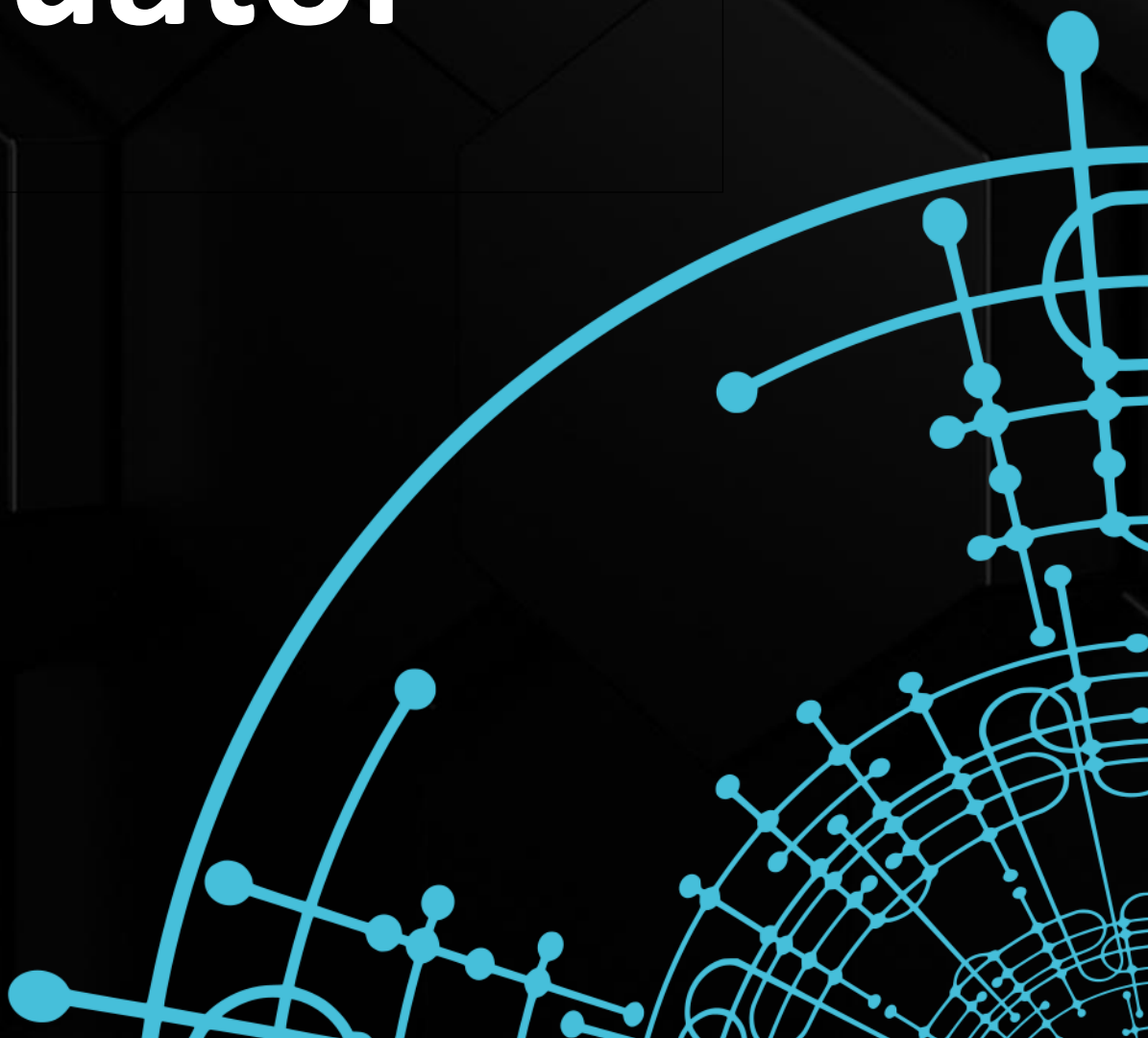
Algorithm Performance Evaluator



Course: Algorithms (CSE 2nd Year) – Computer and Systems Engineering

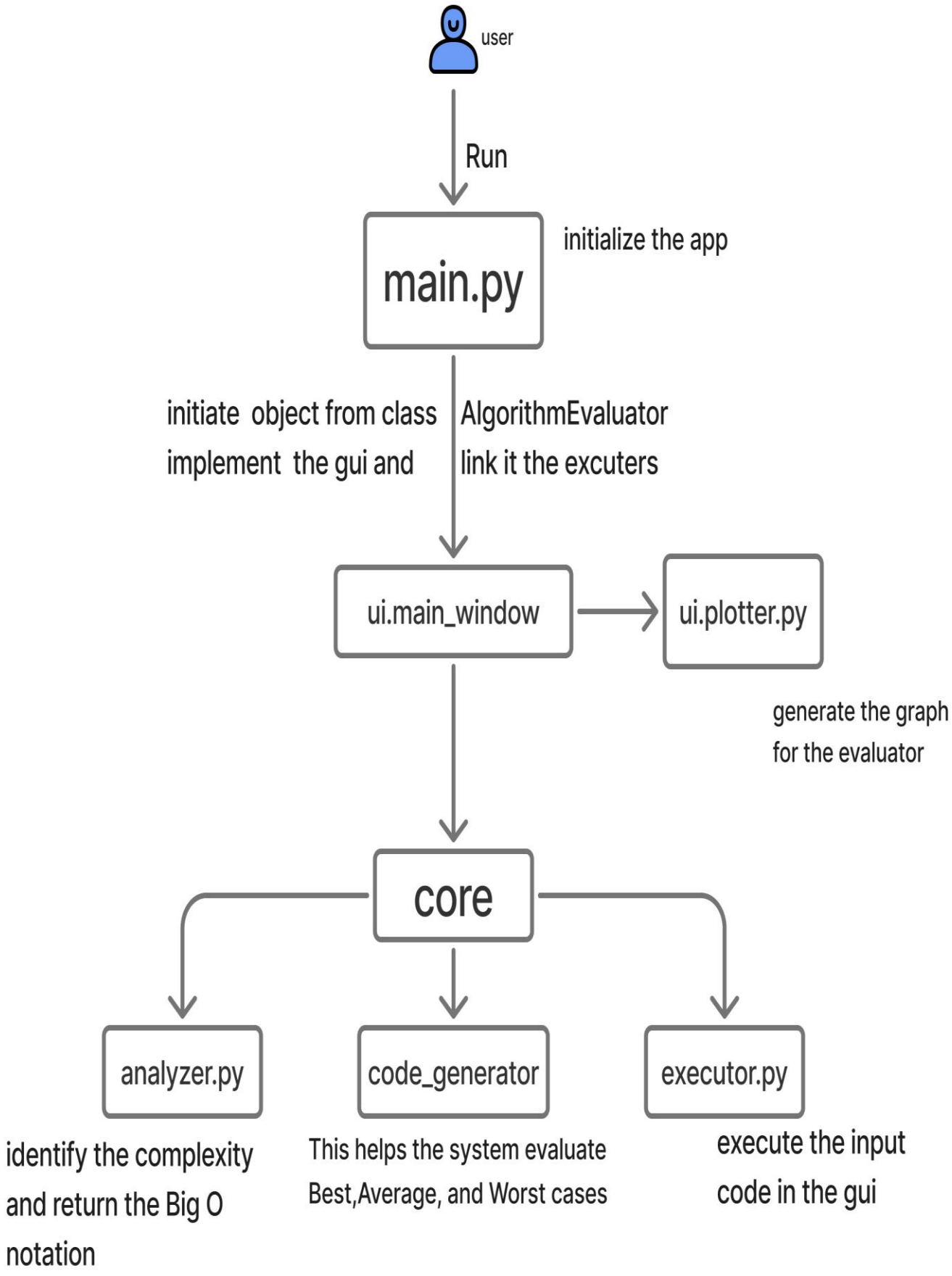
Academic Year: 2025/2026

Subject Teacher: Dr. Hend Gaballah

Teaching Assistant: Eng. Mahmoud Ibrahim

Algorithm Evaluator is a desktop application designed to analyze and evaluate the performance of algorithms in an intuitive and interactive way. Built using CustomTkinter, the application combines modern user interface design with powerful computational analysis to provide a smooth user experience.

The Architecture



Algorithm Performance Evaluator

Algorithm Performance Evaluator

Bubble Sort – O(n²) | Manual | Auto | ▶ Run Analysis

CODE EDITOR Python

```python
def my_algorithm(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

ANALYSIS RESULTS

DETECTED COMPLEXITY

O(n^2)

Quadratic – nested loop pattern detected

Confidence

O(n^2)	100%
O(n)	75%
O(n^3)	69%

BEST	AVG	WORST
12.27 ms	21.48 ms	26.25 ms
Ω(n^2)	Θ(n^2)	O(n^2)

Algorithm Performance Evaluator

Algorithm Performance Evaluator

Linear Search – O(n) | Manual | Auto | ▶ Run Analysis

CODE EDITOR — Python

```python
# EVAL_SIZES: 100, 500, 1000, 5000, 10000
def my_algorithm(arr):
    # Find the maximum value – always scans the ENTIRE array regardless
    # of input order, giving a clean O(n) signal across all three cases.
    # Target-based search was replaced because the target position varied
    # unpredictably across different array sizes, producing noisy curves.
    if not arr:
        return None
    maximum = arr[0]
    for val in arr:
        if val > maximum:
            maximum = val
    return maximum
```

ANALYSIS RESULTS

DETECTED COMPLEXITY

O(n)

Linear – scales directly with input

Confidence

O(n)	100%
O(n log n)	70%
O(n^2)	66%

BEST	AVG	WORST
153.6 µs	94.7 µs	91.2 µs
Ω(n log n)	Θ(n)	O(n)

Algorithm Performance Evaluator

Algorithm Performance Evaluator

Triple Nested Loop – O(n³) | Manual | Auto | ▶ Run Analysis

CODE EDITOR Python

```python
# EVAL_SIZES: 5, 10, 20, 40, 60
# Safe upper bound: n=60 → 60³ = 216,000 ops (runs in milliseconds).
# Do NOT remove or raise EVAL_SIZES — n=1000 would freeze the UI.
def my_algorithm(arr):
    n = len(arr)
    total = 0
    for i in range(n):
        for j in range(n):
            for k in range(n):
                total += arr[i] - arr[j] + arr[k]
    return total
```

ANALYSIS RESULTS

DETECTED COMPLEXITY

O(n^3)

Cubic – triple nested loops detected

Confidence

O(n^3)	100%
O(2^n)	70%
O(n^2)	68%

BEST	AVG	WORST
7.80 ms	7.90 ms	7.96 ms
Ω(n^3)	Θ(n^3)	O(n^3)

Algorithm Performance Evaluator

FPS N/A | GPU 0% | CPU 9% | LAT N/A

Algorithm Performance Evaluator

Load Template... | Manual | Auto | ▶ Run Analysis

CODE EDITOR Python

```python
def my_algorithm(arr):
    if len(arr) == 0:
        return None
    return arr[0]
```

ANALYSIS RESULTS

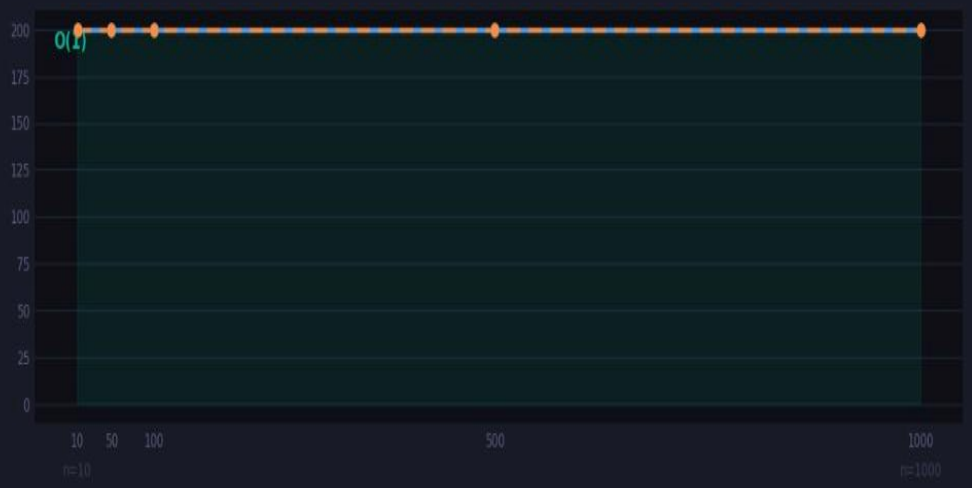
DETECTED COMPLEXITY

O(1)

Constant – independent of input size

Confidence

O(1) 100%



BEST	AVG	WORST
200 ns	200 ns	200 ns
O(1)	O(1)	O(1)

Algorithm Performance Evaluator

Algorithm Performance Evaluator

Merge Sort – O(n log n) | Manual | Auto | ⧖ Processing...

CODE EDITOR Python

```python
# EVAL_SIZES: 100, 500, 1000, 5000, 10000
def my_algorithm(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left  = my_algorithm(arr[:mid])
    right = my_algorithm(arr[mid:])

    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

ANALYSIS RESULTS

DETECTED COMPLEXITY

O(n log n)

Linearithmic – efficient divide & conquer

Confidence

O(n log n) 100%

O(n) 80%

BEST	AVG	WORST
6.99 ms	10.98 ms	7.37 ms
Ω(n log n)	Θ(n log n)	O(n log n)

Algorithm Performance Evaluator

Fibonacci Recursive – O(2^n) | Manual | Auto | ▶ Run Analysis

CODE EDITOR — Python

```python
# EVAL_SIZES: 5, 8, 12, 16, 20
# Safe upper bound: fib(20) needs ~22K Python calls (~1
-2 sec total).
# Do NOT remove or raise EVAL_SIZES – fib(1000) would h
ang indefinitely.
def my_algorithm(arr):
    def fib(num):
        if num <= 1:
            return num
        return fib(num - 1) + fib(num - 2)

    # Use array length as input to fib so the executor'
s size sweep
    # drives the input directly: size=5 → fib(5), size=
20 → fib(20).
    n = len(arr)
    return fib(n)
```

ANALYSIS RESULTS

DETECTED COMPLEXITY

O(2^n)

Exponential – combinatorial blowup

Confidence

O(2^n)		100
O(n^3)		61
O(n^2)		54

BEST	AVG	WORST
1.03 ms	1.03 ms	1.04 ms
Ω(2^n)	Θ(2^n)	O(2^n)

Team Roles & Assignments

Member 1: علي محمد ابوخليل بنداري علي (Team Lead & Systems Integrator) (github: therevenant9342-beep repo manager)

- Assigned File: core/executor.py & make enhancement in ui/main_window.py

- Mission: Build the secure Execution Engine. Connect all the modules together when the team finishes their tasks. Handle GitHub pull requests and merges.

Member 2: محمد باسم السيد السيد احمد جويفل UI/UX Engineer

(Github: Mo-Bassem #5)

- Assigned File: ui/main_window.py

- Mission: Use customtkinter to finalize the interface. We need an Input Panel, a ControlPanel, and a Results Panel. Make sure it matches the professional dark theme of the system.

Member 3: محمد جمال اسماعيل محمود علي Complexity Analyst (Math Engine) (github: dfult856 #9)

- Assigned File: core/analyzer.py
- Mission: Take the timing data (X sizes, Y times) and use numpy to determine if the curve is O(1), O(n), O(n2), etc.

Member 4: يوسف احمد ابراهيم عبدالعال Data Visualization (github: yousef394 #8)

- Assigned File: ui/plotter.py
- Mission: Integrate matplotlib into our customtkinter GUI which draw the graphs that show Runtime across Input Sizes.

Member 5: احمد صلاح عبدالمجيد عبدالحميد testing & Documentation (github: ahmed-salah-1 #7)

- Assigned File: test_algorithms/sample_cases.py & the Final PDF Report.
  - Mission: Write the 6 mandatory test functions (O(1) through O(2n)). Write the final report
  - Bug fixing and enhance the performance of the code.